

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I A Sai Swetha(192319072) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 1: Decision Tree Classification**Objective**

To implement and evaluate a Decision Tree classifier using a dataset such as the **Iris dataset**, preprocess the data, build the tree using Gini Index or Information Gain, and evaluate its performance using accuracy, confusion matrix, and classification report.

CO4-AT1

(a) Loading, preprocessing and splitting the dataset

For this experiment, the **Iris dataset** can be used. It contains 150 observations belonging to three classes:

- Setosa
- Versicolor
- Virginica

The four input features are:

- Sepal length
- Sepal width
- Petal length
- Petal width

There are no missing values in the standard Iris dataset, so no missing-value replacement is required. The target classes are categorical and are automatically represented numerically by `scikit-learn`.

Python Code

Python

Run

```

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
import pandas as pd

# Load dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)

# Add target column
df["target"] = iris.target

# Display first 5 rows
print("First 5 rows:")
print(df.head())

# Display data types
print("\nData types:")
print(df.dtypes)

# Check missing values
print("\nMissing values:")
print(df.isnull().sum())

# Separate input and output
X = df.drop("target", axis=1)
y = df["target"]

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

```

Explanation

`load_iris()` loads the standard Iris dataset.

The dataset is divided into:

- **80% training data**
- **20% testing data**

The training set is used to learn the decision tree, while the testing set is used to determine how well the trained model performs on unseen data.

`random_state=42` ensures that the same train-test split is obtained whenever the program is executed.

`stratify=y` maintains approximately equal proportions of the three classes in both training and testing datasets.

Expected first five rows

Sepal Length	Sepal Width	Petal Length	Petal Width	Target
5.1	3.5	1.4	0.2	0
4.9	3.0	1.4	0.2	0
4.7	3.2	1.3	0.2	0
4.6	3.1	1.5	0.2	0
5.0	3.6	1.4	0.2	0

The target values correspond to:

- 0 → Setosa
- 1 → Versicolor
- 2 → Virginica

(b) Building the Decision Tree

The Decision Tree can be constructed using either:

1. **Gini Index**, or
2. **Information Gain / Entropy**

Here, we use the **Gini Index**.

Gini Index

The Gini impurity is calculated using:

where p_i is the probability of a sample belonging to class i .

A node with only one class has:

which means it is completely pure.

A node containing a mixture of classes has a higher Gini value.

Python Code

Python

Run

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

# Create Decision Tree model
model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)

# Train model
model.fit(X_train, y_train)

# Print tree structure
print("Decision Tree Structure:\n")
print(
    model.tree_
)

# Display feature importance
print("\nFeature Importance:")
for feature, importance in zip(
    X.columns,
    model.feature_importances_
):
    print(f"{feature}: {importance:.4f}")

# Plot decision tree
plt.figure(figsize=(15, 10))

plot_tree(
    model,
    feature_names=iris.feature_names,
```

```
class_names=iris.target_names,  
filled=True  
)  
  
plt.title("Decision Tree Classifier")  
plt.show()
```

Root Node Identification

For the Iris dataset, the root node is typically based on:

Petal width or **petal length**, depending on the exact tree configuration and implementation.

With the standard unrestricted scikit-learn tree and the stated random state, the most important features are generally the **petal measurements**, because they provide the strongest separation between the three Iris classes.

The root node is selected because it produces the greatest reduction in impurity.

For example, a condition may look like:

```
petal width <= 0.8
```

The tree uses this condition to divide the dataset into two groups.

Why is the root node important?

The root node is the first decision made by the classifier. It should provide the best separation of the target classes.

For Iris:

- Setosa generally has very small petals.
- Versicolor has medium-sized petals.
- Virginica has larger petals.

Therefore, petal-related features are highly useful for classification.

(c) Model Evaluation

The assessment specifically requires evaluation using:

- Accuracy score
- Confusion matrix
- Classification report
- At least two misclassified instances
- Comment on performance

CO4-AT1

Python Code

Python

Run

```
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# Predict test data
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Classification report
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=iris.target_names
    )
)

# Find misclassified instances
misclassified = X_test[y_test != y_pred].copy()

misclassified["Actual"] = y_test[y_test != y_pred]
misclassified["Predicted"] = y_pred[y_test != y_pred]
```

```
print("\nMisclassified Instances:")
print(misclassified)
```

Accuracy

The accuracy is:

For example, if 29 out of 30 test samples are classified correctly:

The exact result can vary depending on the train-test split and tree settings.

Confusion Matrix

A typical confusion matrix has the form:

	Predicted		
	Setosa	Versicolor	Virginica
Actual Setosa	10	0	0
Actual Versicolor	0	9	1
Actual Virginica	0	1	9

The diagonal values represent correctly classified samples.

For example:

- Setosa → Setosa: correctly classified
- Versicolor → Versicolor: correctly classified
- Virginica → Virginica: correctly classified

Off-diagonal values represent errors.

Classification Report

The classification report provides:

Precision

It tells us how many samples predicted as a particular class actually belong to that class.

Recall

It indicates how many actual samples of a class were correctly identified.

F1-score

The F1-score combines precision and recall.

Misclassified Instances

The exact misclassified observations should be obtained from the program output rather than invented beforehand.

They can be displayed using:

Python

Run

```
for i in range(len(y_test)):
    if y_test.iloc[i] != y_pred[i]:
        print(
            "Actual:",
            iris.target_names[y_test.iloc[i]],
            "Predicted:",
            iris.target_names[y_pred[i]],
            "Features:",
            X_test.iloc[i].values
        )
```

Typically, classification errors occur between:

Versicolor and Virginica

because their petal dimensions can overlap.

Setosa is usually much easier to distinguish.

Performance Comment

The Decision Tree performs very well on the Iris dataset because the three classes have relatively distinct feature distributions, particularly in terms of petal length and petal width.

The model generally achieves high accuracy. Most predictions fall along the diagonal of the confusion matrix, indicating correct classification.

The small number of errors generally occurs between **Versicolor and Virginica**, whose characteristics overlap more than those of Setosa.

Result

Thus, a Decision Tree classifier was successfully implemented on the Iris dataset. The model was trained using the Gini Index, evaluated using accuracy, confusion matrix and

classification report, and the misclassified instances were identified. The classifier provides high classification performance on the Iris dataset.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, –1 each step, –5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).

(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

Experiment 2: Reinforcement Learning – Q-Learning

Objective

The second experiment requires implementing **Q-Learning** for a **4 × 4 Grid World** and observing how the agent learns an optimal policy. The environment uses four actions—Up, Down, Left and Right—with rewards of **+10 for reaching the goal, –1 for each step, and –5 for an obstacle**. The assessment also requires at least 100 episodes and comparison of Q-values at Episodes 1, 50 and 100.

CO4-AT1

(a) Define the Environment

Consider a 4 × 4 grid.

There are:

- **16 states**
- **4 actions**
- One goal state
- One or more obstacle states

The actions are:

0 → Up

1 → Down
2 → Left
3 → Right

Reward structure

Situation	Reward
Reach goal	+10
Normal movement	-1
Hit obstacle	-5

The negative step reward encourages the agent to reach the goal using a shorter path.

Hyperparameters

We can use:

where:

- **α (learning rate)** controls how strongly new information changes the Q-value.
- **γ (discount factor)** determines the importance of future rewards.
- **ϵ (exploration rate)** determines how often the agent chooses a random action instead of the currently best action.

Q-Learning Equation

The central Q-Learning update equation is:

where:

- s = current state
- a = current action
- r = reward received
- s' = next state
- a' = possible next action
- α = learning rate
- γ = discount factor

Meaning

The agent first looks at its current Q-value.

It then considers:

1. The immediate reward.
2. The best expected future reward.
3. The difference between the old and new estimates.

The Q-value is gradually updated through repeated experience.

Python Implementation

Python

Run

```
import numpy as np
import random
import matplotlib.pyplot as plt

# Grid size
ROWS = 4
COLS = 4

# States: 0 to 15
num_states = ROWS * COLS

# Actions
UP = 0
DOWN = 1
LEFT = 2
RIGHT = 3

actions = [UP, DOWN, LEFT, RIGHT]
num_actions = len(actions)

# Goal and obstacle
goal = 15
obstacles = {5}

# Hyperparameters
alpha = 0.1
```

```

gamma = 0.9
epsilon = 0.1

# Initialize Q-table
Q = np.zeros((num_states, num_actions))

def step(state, action):

    row = state // COLS
    col = state % COLS

    new_row = row
    new_col = col

    if action == UP:
        new_row -= 1

    elif action == DOWN:
        new_row += 1

    elif action == LEFT:
        new_col -= 1

    elif action == RIGHT:
        new_col += 1

    # Boundary condition
    if (
        new_row < 0 or
        new_row >= ROWS or
        new_col < 0 or
        new_col >= COLS
    ):
        return state, -1, False

    next_state = new_row * COLS + new_col

    # Goal
    if next_state == goal:
        return next_state, 10, True

    # Obstacle
    if next_state in obstacles:
        return state, -5, False

    # Normal movement
    return next_state, -1, False

```

Running Q-Learning for 100 Episodes

Python

Run

```
episodes = 100

cumulative_rewards = []

# Store Q-values for two representative states
record_state_1 = 0
record_state_2 = 10

q_episode_1 = None
q_episode_50 = None
q_episode_100 = None

for episode in range(1, episodes + 1):

    state = 0
    total_reward = 0

    for step_count in range(100):

        # Epsilon-greedy action selection
        if random.random() < epsilon:
            action = random.choice(actions)
        else:
            action = np.argmax(Q[state])

        next_state, reward, done = step(state, action)

        # Q-Learning update
        Q[state, action] = Q[state, action] + alpha * (
            reward
            + gamma * np.max(Q[next_state])
            - Q[state, action]
        )

        total_reward += reward
        state = next_state

    if done:
        break
```

```
cumulative_rewards.append(total_reward)

# Record Q-values
if episode == 1:
    q_episode_1 = Q.copy()

elif episode == 50:
    q_episode_50 = Q.copy()

elif episode == 100:
    q_episode_100 = Q.copy()
```

(b) Recording Q-Values

The assessment requires recording Q-values at:

- Episode 1
- Episode 50
- Episode 100

for two representative states.

CO4-AT1

Code

Python

Run

```
print("Q-values at Episode 1")
print("State", record_state_1, q_episode_1[record_state_1])
print("State", record_state_2, q_episode_1[record_state_2])

print("\nQ-values at Episode 50")
print("State", record_state_1, q_episode_50[record_state_1])
print("State", record_state_2, q_episode_50[record_state_2])

print("\nQ-values at Episode 100")
print("State", record_state_1, q_episode_100[record_state_1])
print("State", record_state_2, q_episode_100[record_state_2])
```

Interpretation

At **Episode 1**, the Q-table is initially mostly zero. The agent has very little knowledge about the environment.

At **Episode 50**, the agent has experienced many states and rewards. Q-values associated with useful actions begin to increase.

At **Episode 100**, the Q-values corresponding to actions that lead efficiently toward the goal become relatively high.

Therefore:

Episode 1

↓

Very little knowledge

↓

Random exploration

Episode 50

↓

More experience

↓

Useful actions receive higher Q-values

Episode 100

↓

Strong knowledge of environment

↓

Near-optimal policy

Q-Table Evolution

The Q-table has the structure:

State	Up	Down	Left	Right
0	Q(0,U)	Q(0,D)	Q(0,L)	Q(0,R)
1	Q(1,U)	Q(1,D)	Q(1,L)	Q(1,R)
2	Q(2,U)	Q(2,D)	Q(2,L)	Q(2,R)
...	...	...	...	...

State	Up	Down	Left	Right
15 Q(15,U) Q(15,D) Q(15,L) Q(15,R)				

Initially:

for every state-action pair.

As training progresses, the values change based on the rewards received.

The agent therefore learns which action is more beneficial at each state.

(c) Final Learned Policy

After training, the optimal action can be obtained using:

Python

Run

```
policy = np.argmax(Q, axis=1)

action_symbols = {
    UP: "↑",
    DOWN: "↓",
    LEFT: "←",
    RIGHT: "→"
}

print("Final Learned Policy:\n")

for state in range(num_states):

    if state == goal:
        symbol = "G"

    elif state in obstacles:
        symbol = "X"

    else:
        symbol = action_symbols[policy[state]]

    print(f"{symbol:>3}", end=" ")
```

```
if (state + 1) % COLS == 0:  
    print()
```

A possible learned policy will look conceptually like:

```
→  →  →  ↓  
↑  X  →  ↓  
→  →  →  ↓  
↑  ↑  →  G
```

The exact intermediate arrows can vary depending on exploration, random seed, obstacle placement and training.

Meaning of the policy

Each arrow represents the action having the highest Q-value at that state.

For example:

```
→
```

means the agent has learned that moving right is the most beneficial action.

```
↓
```

means moving downward has the highest expected future reward.

G represents the goal and X represents an obstacle.

Plot Cumulative Reward

Python

Run

```
plt.figure(figsize=(10, 5))  
  
plt.plot(cumulative_rewards)  
  
plt.xlabel("Episode")  
plt.ylabel("Cumulative Reward")  
plt.title("Cumulative Reward per Episode")  
plt.grid(True)
```

```
plt.show()
```

Convergence Behaviour

During the initial episodes, the agent explores the environment.

Because exploration involves random actions, the cumulative reward may be relatively low or negative.

As the number of episodes increases:

- The agent discovers useful paths.
- Q-values become more accurate.
- The agent reaches the goal more frequently.
- Unnecessary movements decrease.
- The cumulative reward generally improves.

Eventually, the reward curve tends to stabilize.

This indicates that the agent's policy has approached a stable solution

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation